

SELinux Basic Theory TASK – 4

- 1. Check the current SELinux status on your Linux server using the `getenforce` and `sestatus` commands, and write down which mode (Enforcing, Permissive, or Disabled) is currently active.**

Ans :

- `getenforce`
- `sestatus`
- `echo "Mode: $(getenforce)"`
- `sestatus | grep "SELinux status\|Current mode"`

- 2. Switch SELinux to Enforcing mode using `setenforce` and verify the change with `getenforce`.

Hint: You may need sudo privileges to change the mode.**

Ans :

- `sudo setenforce 1`
- `getenforce`
- `sestatus`
- `sudo nano /etc/selinux/config`
- `SELINUX=enforcing`

- 3. Simulate an SELinux access denial by creating a new directory (e.g., `/tmp/testapp`), setting its context to a type not allowed for your web server (such as `user_home_t`), and then try to serve a file from it using your web server. Note the error message in `/var/log/audit/audit.log` or `/var/log/messages`.**

Ans :

- `sudo mkdir -p /tmp/testapp`
- `echo "SELinux test file" | sudo tee /tmp/testapp/index.html`

- `sudo yum install httpd -y`
- `sudo dnf install httpd -y`
- `sudo systemctl start httpd`
- `sudo systemctl enable httpd`
- `sudo chcon -R -t user_home_t /tmp/testapp`
- `sudo ln -s /tmp/testapp /var/www/html/testapp`
- `curl http://localhost/testapp/index.html`
- `sudo cat /var/log/audit/audit.log | grep denied`
- `sudo tail -f /var/log/audit/audit.log`
- `sudo cat /var/log/messages | grep AVC`

4. Use the `chcon` command to assign the correct SELinux context (e.g., `httpd_sys_content_t`) to your test directory or file so your web server can access it, then confirm the issue is resolved by reloading the page successfully.

Ans :

- `sudo chcon -R -t httpd_sys_content_t /tmp/testapp`
- `curl http://localhost/testapp/index.html`

5. Document the steps you took to identify and resolve the SELinux access denial, including the commands used, the error log entry, and a brief explanation of how SELinux contexts control access.

Ans :

- SELinux Access Denial: Identification and Resolution
- `sudo mkdir -p /tmp/testapp`
- `echo "SELinux test file" | sudo tee /tmp/testapp/index.html`
- `sudo chcon -R -t user_home_t /tmp/testapp`
- `sudo ln -s /tmp/testapp /var/www/html/testapp`
- `curl http://localhost/testapp/index.html`
- `sudo cat /var/log/audit/audit.log | grep denied`

- Example error entry:
- `avc: denied { read } for pid=1234 comm="httpd"`
- `scontext=system_u:system_r:httpd_t:s0`
- `tcontext=system_u:object_r:user_home_t:s0`
- `tclass=file`
- `sudo chcon -R -t httpd_sys_content_t /tmp/testapp`
- `curl http://localhost/testapp/index.html`
- `ls -Z /tmp/testapp`
- Brief explanation:
- SELinux uses security contexts (labels) to control access. Each process (like httpd) and file has a type. Access is allowed only when SELinux policy permits that process type (`httpd_t`) to access a matching file type (`httpd_sys_content_t`). In this case, access was denied because the file had the incorrect type `user_home_t`, which is not allowed for web server access.